INTERVIEW

Top ReactJS Interview Questions and Answers Of 2024! [Part-2]

By Archana

Oct 03, 2024 | 10 Min Read | 2210 Views
(Last Updated)

In the series of React Interview Questions: Part 1, if you have already checked then you might know that we have stopped at React Routers. So, let's kickstart from the same React Router concept in this part. If by chance, you have missed reading part 1, we suggest you have a look at it (possibly).

Table of contents

1. Most Important React Interview Questions
- What are the components of React Router v4?
- What is the purpose of push() and replace() methods of history?
- How do you programmatically navigate using React Router v4?
- How to get query parameters in React Router v4?
- Why do you get the "Router may have only one child element" warning?
- What is the purpose of the ReactTestUtils

Most Important React Interview Questions

1. What are the components of React Router v4?

React Router v4 provides the below 3 components:

```
I.  <BrowserRouter>
II. <HashRouter>
III. <MemoryRouter>
```

The above components will create browser, hash, and memory history instances. React Router v4 makes the properties and methods of the history instance associated with your router available through the context in the router object.

Before diving into the next section, ensure you're solid on full-stack development essentials like front-end frameworks, back-end technologies, and database management. If you are looking for a detailed Full Stack Development career program, you can join GUVI's Full Stack Development Course with Placement Assistance. You will be able to master the MERN stack (MongoDB, Express.js, React, Node.js) and build real-life projects.

Additionally, if you want to explore JavaScript through a self-paced course, try GUVI's JavaScript certification course.

2. What is the purpose of push() and replace() methods of history?

A history instance has two methods for navigation purposes.
• push()
• replace()
If you think of the history as an array of visited locations, push() will add a new location to the array and replace() will replace the current location in the array with the new one.

3. How do you programmatically navigate using React Router v4?

There are three different ways to achieve programmatic routing/navigation within components.

Using the withRouter() higher-order function:
The withRouter() higher-order function will inject the history object as a prop of the
component. This object provides push() and replace() methods to avoid the usage of
context.

```
import { withRouter } from "react-router-
dom"; // this also works with 'react-router-native'

const Button = withRouter(({ history }) => (
  <button
    type="button"
    onClick={() => {
      history.push("/new-location");
    }}
  >
    {"Click Me!"}
  </button>
));
```

2. Using <Router> component and render props pattern:

The component passes the same props as withRouter(), so you will be able to access the history methods through the history prop.

```
import { Route } from "react-router-dom";

const Button = () => (
  <Route
    render={({ history }) => (
      <button
        type="button"
        onClick={() => {
          history.push("/new-location");
        }}
      >
        {"Click Me!"}
      </button>
    )}
  />
);
```

3. Using context:

This option is not recommended and is treated as an unstable API.

```
const Button = (props, context) => (
  <button
    type="button"
    onClick={() => {
      context.history.push("/new-location");
    }}
  >
    {"Click Me!"}
  </button>
);

Button.contextTypes = {
  history: React.PropTypes.shape({
    push: React.PropTypes.func.isRequired,
  }),
};
```


4. How to get query parameters in React Router v4?

The ability to parse query strings was taken out of React Router v4 because there have been user requests over the years to support different implementations. So the decision has been given to users to choose the implementation they like. The recommended approach is to use the query strings library.

```
const queryString = require("query-string");
const parsed = queryString.parse(props.location.search);
```

You can also use URLSearchParams if you want something native:

```
const params = new URLSearchParams(props.location.search);
const foo = params.get("name");
```

You should use a polyfill for IE11.

5. Why do you get the "Router may have only one child element" warning?

You have to wrap your Route's in a block because is unique in that it renders a route exclusively.

At first, you need to add Switch to your imports:

```
import { Switch, Router, Route } from 'react-router'
```

Then define the routes within the block:

```
<Router>
  <Switch>
    <Route {/* ... */} />
    <Route {/* ... */} />
  </Switch>
</Router>
```

6. What is the purpose of the ReactTestUtils package?

ReactTestUtils are provided in the with-addons package and allow you to perform actions against a simulated DOM for the purpose of unit testing.

7. What is Jest?

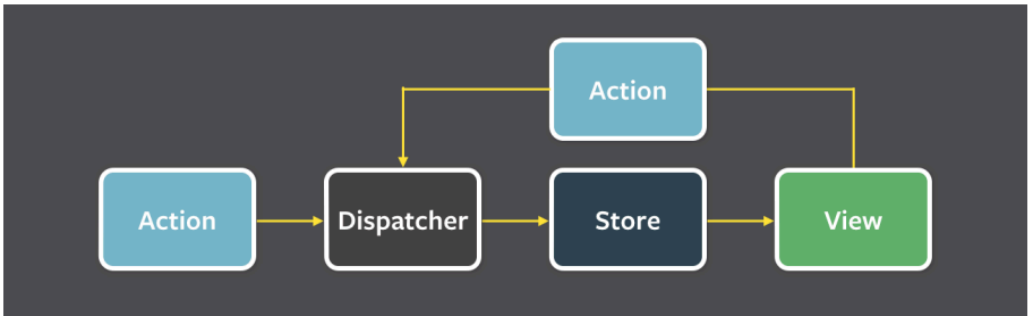
Jest is a JavaScript unit testing framework created by Facebook

based on Jasmine and provides automated mock creation and a jsdom environment. It's often used for testing components.

8. What is flux?

Flux is an application design paradigm used as a replacement for the more traditional MVC pattern. It is not a framework or a library but a new kind of architecture that complements React and the concept of Unidirectional Data Flow. Facebook uses this pattern internally when working with React.

The workflow between dispatcher, stores, and views components with distinct inputs and outputs as follows:



What is Redux?

Redux is a predictable state container for JavaScript apps based on the Flux design pattern. Redux can be used together with React, or with any other view library. It is tiny (about 2kB) and has no dependencies.

9. What are the core principles of Redux?

Redux follows three fundamental principles:

i. Single source of truth: The state of your whole application is stored in an object tree within a single store. The single state tree makes it easier to keep track of changes over time and debug or inspect the application.

ii. State is read-only: The only way to change the state is to emit an action, an object
describing what happened. This ensures that neither the views nor

the network callbacks
will ever write directly to the state.

iii. Changes are made with pure functions: To specify how the state tree is transformed by
actions, you write reducers. Reducers are just pure functions that take the previous state
and an action as parameters, and return the next state.

10. What are the downsides of Redux compared to Flux?

Instead of saying downsides we can say that there are few compromises of using Redux over Flux.

Those are as follows:

i. You will need to learn to avoid mutations: Flux is un-opinionated about mutating data, but Redux doesn't like mutations and many packages complementary to Redux assume you never mutate the state. You can enforce this with dev-only packages like redux-immutable□state-invariant, Immutable.js, or instructing your team to write non-mutating code.

ii. You're going to have to carefully pick your packages: While Flux explicitly doesn't try to
solve problems such as undo/redo, persistence, or forms, Redux has extension points such
as middleware and store enhancers, and it has spawned a rich ecosystem.

iii. There is no nice Flow integration yet: Flux currently lets you do very impressive static type checks which Redux doesn't support yet.

11. Can I dispatch an action in reducer?

Dispatching an action within a reducer is an anti-pattern. Your reducer should be without side effects, simply digesting the action

payload and returning a new state object. Adding listeners and dispatching actions within the reducer can lead to chained actions and other side effects.

12. How to access Redux store outside a component?

You just need to export the store from the module where it created with createStore(). Also, it shouldn't pollute the global window object.

```
store = createStore(myReducer)
```

```
export default store
```

13. What are the drawbacks of MVW pattern?

i. DOM manipulation is very expensive which causes applications to behave slow and
inefficient.
ii. Due to circular dependencies, a complicated model was created around models and views.
iii. Lot of data changes happens for collaborative applications(like Google Docs).
iv. No way to do undo (travel back in time) easily without adding so much extra code.

14. Are there any similarities between Redux and RxJS?

These libraries are very different for very different purposes, but there are some vague similarities. Redux is a tool for managing state throughout the application. It is usually used as an architecture for UIs. Think of it as an alternative to (half of) Angular. RxJS is a reactive programming library. It is usually used as a tool to accomplish asynchronous tasks in JavaScript. Think of it as an alternative to Promises. Redux uses the Reactive paradigm because the Store is reactive. The Store observes actions from a distance, and changes itself. RxJS also uses the Reactive paradigm, but instead of being an architecture, it gives you basic building blocks, Observables, to accomplish this pattern.

15. How to use connect() from React Redux?

You need to follow two steps to use your store in your container:
i. Use mapStateToProps(): It maps the state variables from your store to the props that you
specify.
ii. Connect the above props to your container: The object returned by the mapStateToProps function is connected to the container. You can import connect() from react-redux.

```
class App extends React.Component {
  render() {
    return <div>{this.props.containerData}</div>;
  }
}

function mapStateToProps(state) {
  return { containerData: state.data };
}

export default connect(mapStateToProps)(App);
```

16. What is the difference between React context and React Redux?

You can use Context in your application directly and is going to be great for passing down data to deeply nested components which what it was designed for.

Whereas Redux is much more powerful and provides a large number of features that the Context API doesn't provide. Also, React Redux uses context internally but it doesn't expose this fact in the public API.

17. Why are Redux state functions called reducers?

Reducers always return the accumulation of the state (based on all previous and current actions). Therefore, they act as a reducer of state. Each time a Redux reducer is called, the state and action are passed as parameters. This state is then reduced (or accumulated) based on the action, and then the next state is returned. You could

reduce a collection of actions and an initial state (of the store) on which to perform these actions to get the resulting final state.

18. How to make AJAX request in Redux?

You can use redux-thunk middleware which allows you to define async actions.
Let's take an example of fetching specific account as an AJAX call using fetch API:

```
export function fetchAccount(id) {
  return (dispatch) => {
    dispatch(setLoadingAccountState()); // Show a loading sp
inner
    fetch(`/account/${id}`, (response) => {
      dispatch(doneFetchingAccount()); // Hide loading spinn
er
      if (response.status === 200) {
        dispatch(setAccount(response.json)); // Use a normal
 function to set the received state
      } else {
        dispatch(someError);
      }
    });
  };
}

function setAccount(data) {
  return { type: "SET_Account", data: data };
}
```

19. What is the difference between component and container in React Redux?

Component is a class or function component that describes the presentational part of your
application.
Container is an informal term for a component that is connected to a Redux store. Containers subscribe to Redux state updates and dispatch actions, and they usually don't render DOM elements; they delegate rendering to presentational child components.

20. What is the purpose of the constants in Redux?

Constants allows you to easily find all usages of that specific functionality across the project when you use an IDE. It also prevents you from introducing silly bugs caused by typos – in which case, you will get a ReferenceError immediately.

Normally we will save them in a single file (constants.js or actionTypes.js).

```
export const ADD_TODO = 'ADD_TODO'
export const DELETE_TODO = 'DELETE_TODO'
export const EDIT_TODO = 'EDIT_TODO'
export const COMPLETE_TODO = 'COMPLETE_TODO'
export const COMPLETE_ALL = 'COMPLETE_ALL'
export const CLEAR_COMPLETED = 'CLEAR_COMPLETED'
```

In Redux, you use them in two places:
i. During action creation:
Let's take actions.js:

```
import { ADD_TODO } from './actionTypes';
export function addTodo(text) {
return { type: ADD_TODO, text }
}
```

ii. In reducers:
Let's create reducer.js:

```
import { ADD_TODO } from "./actionTypes";
export default (state = [], action) => {
switch (action.type) {
case ADD_TODO:
     return [
     ...state,
     {
     text: action.text,
     completed: false,
     },
     ];
default:
     return state;
}
};
```

21. How to structure Redux top level directories?

Most of the applications has several top-level directories as below:

i. Components: Used for dumb components unaware of Redux.
ii. Containers: Used for smart components connected to Redux.
iii. Actions: Used for all action creators, where file names correspond to part of the app.
iv. Reducers: Used for all reducers, where files name correspond to state key.
v. Store: Used for store initialization.

This structure works well for small and medium size apps.

22. What is Redux DevTools?

Redux DevTools is a live-editing time travel environment for Redux with hot reloading, action replay, and customizable UI. If you don't want to bother with installing Redux DevTools and integrating it into your project, consider using Redux DevTools Extension for Chrome and Firefox.

23. What are the features of Redux DevTools?

Some of the main features of Redux DevTools are below,
i. Lets you inspect every state and action payload.
ii. Lets you go back in time by cancelling actions.
iii. If you change the reducer code, each staged action will be re-evaluated.
iv. If the reducers throw, you will see during which action this happened, and what the error was.
v. With persistState() store enhancer, you can persist debug sessions across page reloads.

24. What are Redux selectors and why to use them?

Selectors are functions that take Redux state as an argument and return some data to pass to the component.
For example, to get user details from the state:

```
const getUserData = state => state.user.data
```

These selectors have two main benefits,
i. The selector can compute derived data, allowing Redux to store the

minimal possible state
ii. The selector is not recomputed unless one of its arguments changes.

25. What is Redux Form?

Redux Form works with React and Redux to enable a form in React to use Redux to store all of its state. Redux Form can be used with raw HTML5 inputs, but it also works very well with common UI frameworks like Material UI, React Widgets and React Bootstrap.

26. What are the main features of Redux Form?

Some of the main features of Redux Form are:
i. Field values persistence via Redux store.
ii. Validation (sync/async) and submission.
iii. Formatting, parsing and normalization of field values.

27. How to add multiple middlewares to Redux?

You can use applyMiddleware().
For example, you can add redux-thunk and logger passing them as arguments to applyMiddleware():

```
import { createStore, applyMiddleware } from 'redux'
const createStoreWithMiddleware = applyMiddleware(ReduxThunk
, logger)(createStore)
```

28. How to set initial state in Redux?

You need to pass initial state as second argument to createStore:

```
const rootReducer = combineReducers({
  todos: todos,
  visibilityFilter: visibilityFilter
})

const initialState = {
  todos: [{ id: 123, name: 'example', completed: false }]
}

const store = createStore(
  rootReducer,
  initialState
)
```

29. How Relay is different from Redux?

Relay is similar to Redux in that they both use a single store. The main difference is that relay only manages state originated from the server, and all access to the state is used via GraphQL queries (for reading data) and mutations (for changing data). Relay caches the data for you and optimizes data fetching for you, by fetching only changed data and nothing more.

30. What is an action in Redux?

Actions are plain JavaScript objects or payloads of information that send data from your application to your store. They are the only source of information for the store. Actions must have a type property that indicates the type of action being performed.

For example, let's take an action which represents adding a new todo item:

```
{
  type: ADD_TODO,
  text: 'Add todo item'
}
```

31. What is the difference between React Native and React?

React is a JavaScript library, supporting both front end web and being run on the server, for building user interfaces and web applications.

React Native is a mobile framework that compiles to native app components, allowing you to build native mobile applications (iOS, Android, and Windows) in JavaScript that allows you to use React to build your components, and implements React under the hood.

32. What is the difference between Flow and PropTypes?

Flow is a static analysis tool (static checker) which uses a superset of the language, allowing you to add type annotations to all of your code and catch an entire class of bugs at compile time.
PropTypes is a basic type checker (runtime checker) which has been patched onto React. It can't check anything other than the types of the props being passed to a given component. If you want more flexible typechecking for your entire project Flow/TypeScript are appropriate choices.

33. What is React Dev Tools?

React Developer Tools let you inspect the component hierarchy, including component props and state. It exists both as a browser extension (for Chrome and Firefox), and as a standalone app (works with other environments including Safari, IE, and React Native).

The official extensions available for different browsers or environments.
i. Chrome extension
ii. Firefox extension
iii. Standalone app (Safari, React Native, etc)

34. Why is DevTools not loading in Chrome for local files?

If you opened a local HTML file in your browser (file://...) then you must first open Chrome
Extensions and check Allow access to file URLs.

35. What are the advantages of React over Vue.js?

React has the following advantages over Vue.js:
i. Gives more flexibility in large apps developing.
ii. Easier to test.
iii. Suitable for mobile apps creating.
iv. More information and solutions available.

Note: The above list of advantages are purely opinionated and it vary based on the professional experience. But they are helpful as base parameters.

36. What is the difference between React and Angular?

Let's see the difference between React and Angular in a table format.

React	Angular
React is a library and has only the View layer	Angular is a framework and has complete MVC functionality
React handles rendering on the server side	AngularJS renders only on the client side but Angular 2 and above renders on the server side
React uses JSX that looks like HTML in JS which can be confusing	Angular follows the template approach for HTML, which makes code shorter and easy to understand
In React, data flows only in one way and hence debugging is easy	In Angular, data flows both way i.e it has two-way data binding between children and

	parent and hence debugging is often difficult

Note: The above list of differences are purely opinionated and it vary based on the professional experience. But they are helpful as base parameters.

37. Why React tab is not showing up in DevTools?

When the page loads, React DevTools sets a global named **REACT_DEVTOOLS_GLOBAL_HOOK**,
then React communicates with that hook during initialization. If the website is not using React or if React fails to communicate with DevTools then it won't show up the tab.

38. What are Styled Components?

styled-components is a JavaScript library for styling React applications. It removes the mapping between styles and components, and lets you write actual CSS augmented with JavaScript.

39. Can Redux only be used with React?

Redux can be used as a data store for any UI layer. The most common usage is with React and React Native, but there are bindings available for Angular, Angular 2, Vue, Mithril, and more. Redux simply provides a subscription mechanism which can be used by any other code.

40. Do you need to have a particular build tool to use Redux?

Redux is originally written in ES6 and transpiled for production into ES5 with Webpack and Babel. You should be able to use it regardless of your JavaScript build process. Redux also offers a UMD build that can be used directly without any build process at all.

41. What are hooks?

Hooks is a new feature(React 16.8) that lets you use state and other React features without writing a class.

Let's see an example of useState hook example,

```
import { useState } from "react";

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</
button>
    </div>
  );
}
```

42. What are the rules needs to follow for hooks?

You need to follow two rules in order to use hooks,

Call Hooks only at the top level of your react functions. i.e, You shouldn't call Hooks inside
loops, conditions, or nested functions. This will ensure that Hooks are called in the same
order each time a component renders and it preserves the state of Hooks between multiple useState and useEffect calls.
ii. Call Hooks from React Functions only. i.e, You shouldn't call Hooks from regular JavaScript functions.

43. What are the differences between Flux and Redux?

Below are the major differences between Flux and Redux

Flux	**Redux**
State is mutable.	State is immutable
The Store contains both state and	The Store and change logic are separate.

change logic.	
There are multiple stores exist.	There is only one store exist.
All the stores are disconnected and flat.	Single store with hierarchical reducers.
It has a singleton dispatcher.	There is no concept of dispatcher.
React components subscribe to the store.	Container components uses connect function.

44. What are the benefits of React Router V4?

Below are the main benefits of React Router V4 module,

i. In React Router v4(version 4), the API is completely about components. A router can be visualized as a single component(<Browser Router>) which wraps specific child router components(<Route>).

ii. You don't need to manually set history. The router module will take care history by wrapping routes with<Browser Router> component.

iii. The application size is reduced by adding only the specific router module(Web, core, or native)

45. What is the behavior of uncaught errors in react 16?

In React 16, errors that were not caught by any error boundary will result in unmounting of the whole React component tree. The reason behind this decision is that it is worse to leave corrupted UI in place than to completely remove it.

For example, it is worse for a payments app to display a wrong amount than to render nothing.

46. What are the possible return types of render method?

Below are the list of following types used and return from render method,

i. React elements: Elements that instruct React to render a DOM node. It includes html elements such as and user defined elements.

ii. Arrays and fragments: Return multiple elements to render as Arrays and Fragments to wrap multiple elements

iii. Portals: Render children into a different DOM subtree.

iv. String and numbers: Render both Strings and Numbers as text nodes in the DOM

v. Booleans or null: Doesn't render anything but these types are used to conditionally render content.

47. What is the main purpose of constructor?

The constructor is mainly used for two purposes,

To initialize local state by assigning object to this.state

For binding event handler methods to the instance For example, the below code covers both the above cases,

```
constructor(props) {
  super(props);
  // Don't call this.setState() here!
  this.state = { counter: 0 };
  this.handleClick = this.handleClick.bind(this);
}
```

48. Is it mandatory to define constructor for React component?

No, it is not mandatory. i.e, If you don't initialize state and you don't bind methods, you don't need to implement a constructor for your React component.

49. What are default props?

The defaultProps are defined as a property on the component class to set the default props for the class. This is used for undefined props, but not for null props.

For example, let us create color default prop for the button component,

```
class MyButton extends React.Component {
  // ...
}

MyButton.defaultProps = {
  color: "red",
};
```

If props.color is not provided then it will set the default value to 'red'. i.e, Whenever you try to access the color prop it uses default value

```
render() {
  return <MyButton /> ; // props.color will be set to red
}
```

Note: If you provide null value then it remains null value.

50. What is the browser support for react applications?

React supports all popular browsers, including Internet Explorer 9 and above, although some polyfills are required for older browsers such as IE 9 and IE 10. If you use es5-shim and es5-sham polyfill then it even support old browsers that doesn't support ES5 methods.

51. What is the benefit of strict mode?

The strict mode will be helpful in the below cases

i. Identifying components with unsafe lifecycle methods.
ii. Warning about legacy string ref API usage.
iii. Detecting unexpected side effects.
iv. Detecting legacy context API.
v. Warning about deprecated findDOMNode usage

52. What is the difference between Real DOM and Virtual DOM?

Below are the main differences between Real DOM and Virtual DOM,

Real DOM	Virtual DOM
Updates are slow	Updates are fast
DOM manipulation is very expensive.	DOM manipulation is very easy
You can update HTML directly.	You Can't directly update HTML
It causes too much of memory wastage.	There is no memory wastage.
Creates a new DOM if element updates	It updates the JSX if element update

53. How to add Bootstrap to a react application?

Bootstrap can be added to your React app in a three possible ways,
i. Using the Bootstrap CDN: This is the easiest way to add bootstrap. Add both bootstrap CSS and JS resources in a head tag.

ii. Bootstrap as Dependency: If you are using a build tool or a module bundler such as
Webpack, then this is the preferred option for adding Bootstrap to your React application
npm install bootstrap

iii. React Bootstrap Package: In this case, you can add Bootstrap to our React app is by using a package that has rebuilt Bootstrap components to work particularly as React components.

Below packages are popular in this category,
a. react-bootstrap
b. reactstrap

54. Can you list down top websites or applications using react as front end framework?

Below are the top 10 websites using React as their front-end framework,
I. Facebook
II. Uber
III. Instagram
IV. WhatsApp
V. Khan Academy
VI. Airbnb
VII. Dropbox

VIII. Flipboard
IX. Netflix
X. PayPal

55. Is it recommended to use CSS In JS technique in React?

React does not have any opinion about how styles are defined but if you are a beginner then good starting point is to define your styles in a separate *.css file as usual and refer to them using className. This functionality is not part of React but came from third-party libraries. But If you want to try a different approach(CSS-In-JS) then styled-components library is a good option.

56. How do you access imperative API of web components?

Web Components often expose an imperative API to implement its functions. You will need to use a ref to interact with the DOM node directly if you want to access imperative API of a web component.

But if you are using third-party Web Components, the best solution is to write a React component that behaves as a wrapper for your Web Component.

57. What are typical middleware choices for handling asynchronous calls in Redux?

Some of the popular middleware choices for handling asynchronous calls in Redux eco system are

Redux Thunk, Redux Promise, Redux Saga.

58. Do browsers understand JSX code?

No, browsers can't understand JSX code. You need a transpiler to convert your JSX to regular Javascript that browsers can understand. The most widely used transpiler right now is Babel.

59. Describe about data flow in react?

React implements one-way reactive data flow using props which reduce boilerplate and is easier to understand than traditional two-way data binding.

60. What is react scripts?

The react-scripts package is a set of scripts from the create-react-app starter pack which helps you kick off projects without configuring. The react-scripts start command sets up the development environment and starts a server, as well as hot module reloading.

61. What are the features of create react app?

Below are the list of some of the features provided by create react app.
I. React, JSX, ES6, Typescript and Flow syntax support.
II. Autoprefixed CSS
III. CSS Reset/Normalize
IV. A live development server
V. A fast interactive unit test runner with built-in support for coverage

reporting
VI. A build script to bundle JS, CSS, and images for production, with hashes and sourcemaps
VII. An offline-first service worker and a web app manifest, meeting all the Progressive Web App criteria.

Kickstart your Full Stack Development journey by enrolling in GUVI's certified **Full Stack Development Course** with Placement Assistance where you will master the MERN stack (MongoDB, Express.js, React, Node.js) and build interesting real-life projects. This program is crafted by our team of experts to help you upskill and assist you in placements.

Alternatively, if you want to explore JavaScript through a self-paced course, try GUVI's JavaScript course.

Wrapping Up

We hope that these React Interview questions are good set of questions for your Interview preparations. Stay abreast with the latest technologies with tech career courses in Zen Class from GUVI.

Have a query? Why don't you drop your queries in comments below. To get connected to our Experts, you can leave your contact number with us.

About the Author

Archana

A traveler, and explorer, Archana is an active writer at GUVI. You can usually find her with a book/kindle binge-reading (with an affinity to the mystery/humor).

Connect with me @

Did you enjoy this article?

Name

Email

Rate this Article ☆ ☆ ☆ ☆ ☆

Enter Your Comment

Submit My Comment

Learn

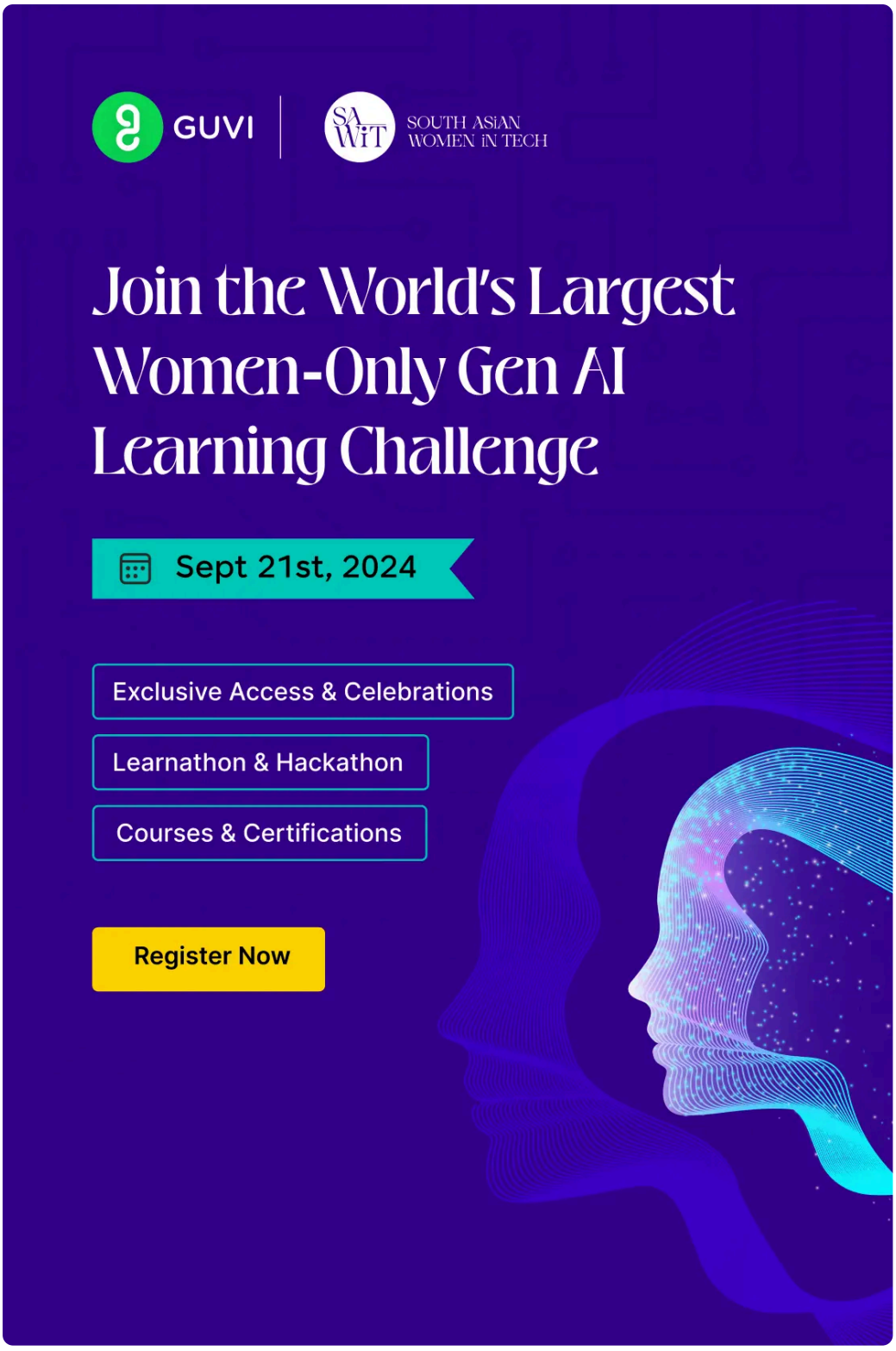
Full Stack Development

in your favourite programming stack

- MERN, MEAN, Java, Python & more
- Mentors from Global Companies
- Build Portfolio with 20+ Projects

Enroll Now

*Learn in english, தமிழ், ಕನ್ನಡ & हिंदी



Get In Touch For Details!

Name*

Email ID*

Phone Number*

+91

Select your interested program*

Choose Program

Get In Touch Request Information

Recommended Courses

Career Programs | Micro Courses

Java Full Stack Development Course

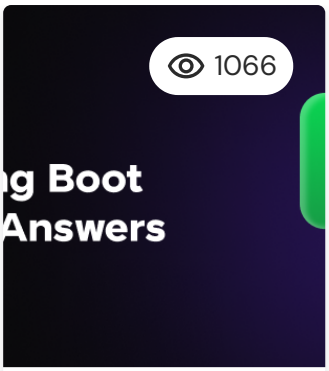
Available in

ENGLISH TAMIL

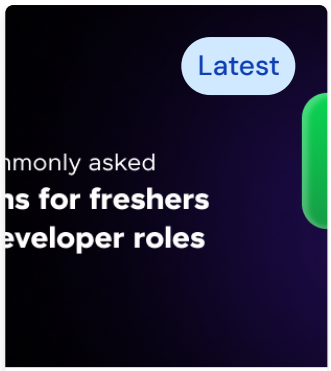
Know More >>

Similar Articles

How to Confidently Explain Test Automation Frameworks in Any



Ace Your Interview: Top 50+ Spring Boot Questions and Answers



100 Commonly Asked Interview Questions for Java

Blog Categories

Data Science

Artificial Intelligence and Machine Learning

Blockchain

Cloud Computing

Cyber Security

Digital Marketing

Full Stack Development

Interview Questions

General Interview Questions

Java Interview Questions

Python Interview Questions

SQL Interview Questions

Selenium Interview Questions

Data Science Interview Questions

Salary blog

UI/UX Designer Salary

Data Scientist Salary

Full-Stack Developer Salary

Motion Graphics Designer Salary

Cloud Computing Engineer Salary

Digital Marketer Salary

About us

Our Story

Careers

Refund Policy

FAQs

Contact Us

GUVI

Refer & Earn

GUVI – India's Pioneering Vernacular EdTech, incubated by IIT-M, IIM-A, and a prestigious part of the HCL group. Empowering over 2.5 million global learners with top-tier educational solutions through a vernacular approach, we have revolutionized global tech education with strategic partnerships, including 'Google for Education,' AICTE,

AWS, Google Cloud, IIT-M Pravartak, UiPath, and NASSCOM. GUVI is your trusted guide to inclusive and accredited learning experiences.

Follow us on

Table of contents